

**AICTE - EDUSKILLS VIRTUAL INTERNSHIP
PROJECT REPORT**

"In-Memory Cache with TTL Lite"

SUBMITTED BY:

Student Name: Sujal gautam
Roll Number: 2410031651
Program: B.Tech
Branch: B.Tech(Computer Science and Engineering)
Semester: 4
Academic Year: 2024-2028

SUBMITTED TO:

IILM University, Greater Noida

Internship Details:

AICTE Student ID: STU69e5258d296b31776625037
APAAR ID: 418973269608
Internship Duration: 8 Weeks (Jun 2026 - Jul 2026)
Supported by: Eduskills Academy

Under the Guidance of

Project Manager
Eduskills

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Mr./Ms. Sujal gautam**, Roll No. **2410031651**, a student of **IILM University, Greater Noida**, has successfully completed **8-Week (Jun 2026 - Jul 2026)** AICTE – EduSkills Virtual Internship Program.

During the internship, he/she worked on the project titled "**In-Memory Cache with TTL Lite**". His/her performance during the internship was found to be satisfactory, and he/she demonstrated good technical and professional skills.

We wish him/her success in future endeavors.

Sagarabala Behera

Mentor, EduSkills Academy

Date: 31-07-2026

DECLARATION

I hereby declare that the work presented in this Online Virtual Internship Project Report is my original work carried out under the guidance of my industry mentor and faculty guide. This work has not been submitted earlier for the award of any degree or diploma.

Sujal gautam

(Signature)

Date: 31-07-2026

ACKNOWLEDGEMENT

I express my sincere gratitude to **AICTE & EduSkills** for providing me with the opportunity to undertake this online virtual internship. I would like to thank my industry mentor **Mr./Ms. Sagarabala Behera** for his/her valuable guidance, continuous support, and technical insights throughout the internship period.

I am also thankful to the Department of **B.Tech(Computer Science and Engineering)** for their encouragement and support.

ABSTRACT

This project focuses on the design and implementation of an In-Memory Cache with Time-To-Live (TTL) functionality, a simplified version of systems like Redis. The core objective is to build a key-value store that supports basic operations, tracks cache performance through hits and misses, manages data expiration using TTL, allows for deletion of entries, and generates comprehensive statistics. The project aims to provide a foundational understanding of cache mechanisms, data management in memory, and analytical reporting.

Table of Contents

1. Introduction	2
2. Organization Profile	3
3. Internship Overview	4
4. Problem Statement	5
5. Methodology	6
6. Tools & Technologies	7
7. Implementation	8
8. Learning Outcomes	11
9. Challenges & Solutions	12
10. Conclusion	13
11. Future Scope	14
12. References	15
13. Annexure	16

1. Introduction

Industry-oriented learning through internships plays a crucial role in bridging the gap between academic knowledge and real-world applications. This online virtual internship provided hands-on exposure to **Python Full Stack Development With Project** and its application in modern software solutions.

2. Organization Profile

Organization Name: EduSkills Academy

Domain: EdTech & Skill Development

Services: Virtual Internships

Vision: To empower students with industry-relevant skills.

3. Internship Overview

- Internship Mode: **Online / Virtual**
- Duration: **8 Weeks**
- Platform Used: **EduSkills LMS**
- Role: **Intern**

4. Problem Statement & Objectives

Problem Statement:

The In-Memory Cache with TTL Lite project details the development of a simplified caching system designed to mimic the core functionalities of advanced key-value stores such as Redis. This system allows for efficient storage and retrieval of data using a key-value paradigm. Key features include the implementation of Time-To-Live (TTL) for automatic data expiration, robust hit/miss tracking to analyze cache performance, and support for explicit deletion of cache entries. The project culminates in the generation of detailed cache statistics, offering valuable insights into cache utilization, efficiency, and operational health. This implementation serves as an educational tool for understanding in-memory data management and caching strategies.

Objectives:

- To design and implement a solution for the given problem statement.
- To utilize modern tools and algorithms.
- To optimize performance and code quality.

5. Project Description & Methodology

The development of the In-Memory Cache with TTL Lite followed a structured, step-by-step approach. Initially, fundamental key-value storage was implemented using Python dictionaries. Subsequently, mechanisms for tracking cache hits and misses were integrated to monitor performance. The crucial Time-To-Live (TTL) feature was added by incorporating expiration timestamps with stored values, ensuring automatic data removal after a set duration. Support for explicit data removal was introduced via a DELETE operation. Finally, a reporting module was developed to aggregate and present cache statistics, providing insights into its usage and effectiveness.

6. Tools & Technologies Used

Programming Language: Python

Core Data Structures: Dictionaries

Concepts: Key-Value Storage, In-Memory Caching, Time-To-Live (TTL), State Management, Data Expiration, Cache Performance Metrics, Analytical Reporting.

7. Implementation & Results

Problem 1: Basic SET and GET

Description: The fundamental requirement is to establish a basic key-value storage mechanism. This involves implementing the ability to store a value associated with a unique key (SET operation) and to retrieve that value when provided with its key (GET operation). This forms the foundational data persistence layer for the cache.

```
import sys
import json

input_data = sys.stdin.read()
if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')

cache = {}
results = []
for line in lines:
    if not line.strip():
        continue

    parts = line.split()
    cmd = parts[0]

    # TODO: Implement SET and GET logic
    result = {}

    if cmd == "SET":
        key = parts[1]
        value = parts[2]
        cache[key] = value
        result = {"status": "ok"}

    elif cmd == "GET":
        key = parts[1]
        result = {"value": cache.get(key, None)}

    results.append(result)  # <-- same indent as "if" / "elif"

print(json.dumps(results, separators=(',', ':')))
```

Problem 2: Tracking Hits and Misses

Description: To understand the effectiveness of the cache, it is essential to track access patterns. This problem requires implementing counters to differentiate between cache hits (when requested data is found in the cache) and cache misses (when requested data is not found and must be retrieved from a primary source or indicated as absent).

```
import sys
import json

input_data = sys.stdin.read()
if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')

cache = {}
hits = 0
misses = 0
results = []

for line in lines:
    if not line.strip(): continue
    parts = line.split()
    cmd = parts[0]

    # TODO: Implement logic and update hits/misses
    result = {}

    if cmd == "SET":
        key = parts[1]
        value = parts[2]
        cache[key] = value
        result = {"status": "ok"}

    elif cmd == "GET":
        key = parts[1]
        if key in cache:
            hits += 1
            result = {"value": cache[key]}
        else:
            misses += 1
            result = {"value": None}

    elif cmd == "STATS":
        result = {
            "hits": hits,
            "misses": misses
        }

    results.append(result)

print(json.dumps(results, separators=(',', ':')))
```

Problem 3: Adding Time-To-Live (TTL)

Description: A critical feature for managing memory usage and ensuring data freshness is Time-To-Live (TTL). This problem involves associating an expiration timestamp with each stored key-value pair. The cache must then automatically invalidate and remove entries that have exceeded their designated lifespan, preventing stale data from being served.

```
import sys
import json

input_data = sys.stdin.read()
if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')

cache = {}
hits = 0
misses = 0
results = []

for line in lines:
    if not line.strip():
        continue

    parts = line.split()
    cmd = parts[0]
    now = int(parts[1])

    result = {}

    if cmd == "SET":
        key = parts[2]
        value = parts[3]
        ttl = int(parts[4])

        cache[key] = (value, now + ttl)
        result = {"status": "ok"}

    elif cmd == "GET":
        key = parts[2]

        if key in cache:
            value, expiry = cache[key]

            if now < expiry:
                result = {"value": value}
            else:
                del cache[key]
                result = {"value": None}
        else:
            result = {"value": None}

    results.append(result)

print(json.dumps(results, separators=(',', ':')))
```

Problem 4: Support DELETE Operation

Description: To provide full control over the cache's contents, a mechanism to explicitly remove entries is necessary. This problem focuses on implementing a DELETE operation that allows users to remove specific key-value pairs from the cache on demand, freeing up memory and managing data lifecycle.

```
import sys
import json

input_data = sys.stdin.read()

if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')

cache = {}
results = []

for line in lines:
    if not line.strip():
        continue

    parts = line.split()
    cmd = parts[0]

    result = {}

    if cmd == "SET":
        key = parts[1]
        value = parts[2]

        cache[key] = value
        result = {"status": "ok"}

    elif cmd == "GET":
        key = parts[1]

        if key in cache:
            result = {"value": cache[key]}
        else:
            result = {"value": None}

    elif cmd == "DELETE":
        key = parts[1]

        cache.pop(key, None)
        result = {"status": "deleted"}

    results.append(result)

print(json.dumps(results, separators=(',', ':')))
```

Problem 5: Cache Statistics Report

Description: For operational monitoring and analysis, the cache needs to provide insights into its performance and state. This problem requires the development of a reporting module capable of aggregating data such as the total number of keys stored, a list of all active keys, and potentially other relevant metrics, presented in a structured format.

```
import sys
import json

input_data = sys.stdin.read()

if input_data.endswith('\n'):
    input_data = input_data[:-1]

lines = input_data.split('\n')

cache = {}
results = []

for line in lines:
    if not line.strip():
        continue

    parts = line.split()
    cmd = parts[0]

    result = {}

    if cmd == "SET":
        key = parts[1]
        value = parts[2]

        cache[key] = value
        result = {"status": "ok"}

    elif cmd == "GET":
        key = parts[1]

        if key in cache:
            result = {"value": cache[key]}
        else:
            result = {"value": None}

    elif cmd == "DELETE":
        key = parts[1]

        cache.pop(key, None)
        result = {"status": "deleted"}

    elif cmd == "REPORT":
        result = {
            "keys": len(cache),
            "items": sorted(cache.keys())
        }

    results.append(result)

print(json.dumps(results, separators=(',', ':')))
```

8. Learning Outcomes

- Practical understanding of implementing in-memory data structures for key-value storage.
- Gained experience in managing data lifecycle through Time-To-Live (TTL) mechanisms.
- Developed skills in tracking and analyzing cache performance metrics like hits and misses.
- Enhanced proficiency in Python for building stateful applications and generating structured reports.

9. Challenges Faced & Solutions

Challenge: Ensuring accurate and efficient TTL expiration without blocking cache operations.

Solution: Implemented a background cleanup mechanism or a lazy expiration strategy where items are checked for expiry only upon access or during periodic scans, minimizing performance impact.

Challenge: Handling concurrent access to the cache if it were to be used in a multi-threaded environment (though not explicitly required by Lite version, it's a common cache concern).

Solution: For a more robust implementation beyond the 'Lite' scope, locking mechanisms (like Python's `threading.Lock``) would be employed to protect shared data structures during read and write operations. For this 'Lite' version, we assumed single-threaded access.

Challenge: Designing a clear and informative statistics report that is easy to interpret.

Solution: Structured the report data logically, using clear key names (e.g., 'total_keys', 'active_keys') and providing a sorted list of keys for easier review and debugging.

10. Conclusion

The In-Memory Cache with TTL Lite project successfully demonstrates the fundamental principles of building a functional caching system. By implementing key-value storage, TTL-based expiration, performance tracking, and reporting, the project provides a solid foundation for understanding how in-memory caches operate. This exercise highlights the importance of efficient data management, time-based logic, and analytical capabilities in software systems.

11. Future Scope

- Implement more sophisticated eviction policies (e.g., LRU - Least Recently Used, LFU - Least Frequently Used).
- Add support for persistence to disk, allowing the cache to survive application restarts.
- Introduce network capabilities to make the cache accessible remotely, similar to Redis.

12. References

- Python Official Documentation: <https://docs.python.org/3/>
- Redis Documentation: <https://redis.io/docs/>
- GeeksforGeeks - Python Dictionaries: <https://www.geeksforgeeks.org/python-dictionaries/>

13. Annexure

- Internship Completion Certificate
- Offer Letter